



How to help evolve the Shoggoth

A plain guide to choosing useful work, volunteering for a Fiat run, and leaving evidence a maintainer can inspect.

**ONE OFFER. ONE QUEUE.
ONE VISIBLE DELIVERY.**

Start with the earliest Wave that still has open issues.

Name the issue. Then run Fiat.

You do not need to understand the whole suite. You need one bounded job and an exact issue URL.

A USEFUL OPENING

I can take this issue through a Fiat run:

`github.com/wildcat-finance/skills/issues/323`

01 Check Open, unassigned, and no visible issue branch or pull request.

02 Bind Pass the exact issue URL when invoking Fiat.

03 Deliver Study, runbook, implementation, audit, prose, push, integration.

04 Review A maintainer sees the change and the evidence around it.

WORKED EXAMPLE

An external contributor has already done it

On 22 August 2026, an external contributor used '/fiat how do i help evolve you'. The run delivered issue #438 through PR #445, added issue-aware branch names, fixed a malformed issue-URL case during audit, and published Fiat 5.10.1 with Hexaameron 1.5.4.

PRESENT BEHAVIOUR

Why the issue matters

Friendly wording can accidentally suggest a frontier job. An issue URL names the actual work. The proposed selector on the next pages makes the queue explicit too.

Say which queue you mean.



Three lanes. No mind-reading.

The command spelling is illustrative. These forms are proposed, not live.

/FIAT VOLUNTEER --LANE WAVE

Wave

Default for a bare volunteer offer. Find the earliest Wave that still has open issues. On the dated snapshot, that is Wave 3 with issues #323 through #328. Apply eligibility inside that Wave. If none is eligible, stop and explain why.

/FIAT VOLUNTEER --LANE FRONTIER --SKILL FIAT

Frontier

An explicit request to advance a skill's held next improvement. The owning evolution ledger and maturity gate decide whether the job is available. The word 'evolve' alone should never select this lane.

/FIAT VOLUNTEER --LANE MAINTENANCE

Maintenance

Bounded upkeep or planning: refresh a Horos boundary, take a fresh issue census, or produce a Kronos-style ranking and proposed Wave split. Useful maintenance does not pretend it advanced a frontier.

SELECTION ORDER

1

Exact issue URL wins

2

Explicit lane stays in its lane

3

Bare offer starts at the earliest open Wave

4

No eligible issue means stop, not silent fallback

Visible steps. Recorded evidence. One delivery.

The job supplies the work. The framework supplies order, specialist tools, checks, and a durable trail.

Promise Machine

The shared law: claims and next actions stay inside the evidence boundary.

Domain skills

Specialists answer the narrow subject question. Hex selects only what the job needs.

Phase skills

Travelling disciplines specify, guard, observe, measure, fix causes, and record decisions.

Hex + Fiat

Hex chooses the toolset. Fiat moves one receipted step at a time.

ONE FIAT RUN

STUDY

RUNBOOK

IMPLEMENT

AUDIT

PROSE

PUSH

INTEGRATE

THE MISSING PIECE

Make volunteer intent and the public claim visible.

Issue #447 asks where the intent packet should live, which public signal proves someone is working the issue, when a new census is required, and how suffixed Waves such as 5b and 9b should be ordered.

DISCUSS THE SELECTOR

wildcat-finance/skills #447

Stopping early still helps.

Fiat commits its thinking before it commits any code, so a run that stops partway still leaves work somebody else can start from.

COMMITTED FIRST

Study

The problem, the options, the chosen design and the risks. The next person can argue with that design on the record, or build from it.

THE EXPENSIVE PART

Runbook

The work already cut into discrete steps with provable exits. The next person can take one step without re-deciding the shape of the job.

ONE BRANCH EACH

Pushed steps

Every finished step on its own issue-linked branch and stacked pull request, with its tests and audit log. The next person continues from the last one.

PUSH BEFORE YOU STOP

The ledger is local.

Controller state and receipts live in `.hexaameron/`, which is untracked. Resuming is a local operation, so only what you pushed reaches anybody else. Say where you stopped, the way a finished run names what it left under Carried forward.

Nobody has infinite tokens.

You run Fiat under your own account, and what returns to the repository is branches rather than tokens. There is no pool to draw on, so a study, a runbook or one audited step is a real contribution at a size you choose.